Face Recognition using Linear Algebra Techniques

Final Report

Ukrainian Catholic University

Team

Taras Korchevyi
Maksym Marych
Matvii Shumylovych

April 2025

1 Project description

Face recognition is one of the core tasks in computer vision, involving the identification or verification of individuals based on facial features. The challenge lies in building a robust and efficient model that performs well across different lighting conditions, orientations, and facial expressions while maintaining reasonable computational costs.

Our project explores various methods for face recognition with a strong emphasis on **linear algebra**. We focus particularly on how dimensionality reduction techniques—especially **Principal Component Analysis (PCA)** and **Linear Discriminant Analysis (LDA)**—can enhance model efficiency and interpretability, while also comparing these methods with modern deep learning approaches like **Convolutional Neural Networks (CNNs)** and **ResNet50+kNN**. To evaluate the robustness of these methods, we compared results across two distinct datasets:

- **Dataset 1 (Complex Dataset):** Contains 10 individuals, with approximately 100 photos per person, totaling 1,000 images. This dataset is challenging due to variations in head orientation, photos taken across different life periods (e.g., individuals may wear glasses, appear older, or have dyed hair), and diverse lighting conditions.
- **Dataset 2 (The ORL Database of Faces):** Comprises 41 individuals, with 10 photos per person, totaling 410 images. Photos were taken in a controlled setting at the same time, resulting in less variation compared to the first dataset.

Our team aimed to investigate how common algorithms perform on these two datasets and compare their effectiveness. We hypothesized that deep learning methods (CNN, ResNet50+kNN) would perform better on the first dataset due to the larger training data size and variability, while linear algebra-based methods (e.g., PCA, LDA) would excel on the second dataset, as the photos in the ORL Database are more similar to each other.

2 Problem Solution

2.1 Is the Choice of the Solution Method Justified, Its Pros and Cons Discussed?

The selection of methods—PCA (using covariance matrix and SVD), LDA, PCA+LDA, NMF, CNN, and ResNet50+kNN—is justified as it encompasses a spectrum of techniques from traditional linear algebra to modern deep learning, enabling a thorough comparison. Each method addresses different aspects of face recognition, balancing accuracy, computational efficiency, and interpretability. Below, we discuss the pros and cons of each method to validate their inclusion:

Method	Pros	Cons
PCA	Efficient dimensionality reduction, easy to implement, highly interpretable	Does not use class labels, less effective for classification
LDA	Maximizes class separability, optimized for classification	Assumes Gaussian distribution, requires labeled data
PCA+LDA	Combines PCA's dimensionality reduction with LDA's classification optimization	Potential information loss from PCA's unsupervised reduction
SVD	Numerically stable, efficient for PCA computation	Computationally expensive for large matrices
NMF	Parts-based representation, intuitive for facial features	Less accurate, computationally intensive
CNN	High accuracy, robust to variations	Requires large datasets, high computational cost, less interpretable
ResNet50+kNN	Leverages pre-trained deep features, high accuracy on complex data	High computational cost, less interpretable, requires tuning

These methods were chosen to test our hypothesis that deep learning methods would excel on the complex dataset due to its size and variability, while linear algebra methods would perform better on the simpler ORL dataset.

2.2 Are the LA Methods Explained in Sufficient Detail?

The linear algebra methods are explained with detailed mathematical formulations, including the use of power iteration for eigenvalue computations where applicable.

2.2.1 Principal Component Analysis (PCA)

PCA reduces dimensionality by projecting data onto a subspace that maximizes variance. For the ORL dataset, images are resized to 64×64 pixels and flattened into vectors of size 4096×1, forming a data matrix $X \in \mathbb{R}^{4096 \times 410}$:

$$X_{4096\times 410} = \begin{bmatrix} | & | & | & \cdots & | \\ x_1 & x_2 & x_3 & \cdots & x_{410} \\ | & | & | & \cdots & | \end{bmatrix}$$

For the Complex Dataset, the data matrix is $X \in \mathbb{R}^{4096\times 1000}$.
Steps:

1. **Mean Centering:** Compute the mean vector $\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$, where n is the number of samples (410 for ORL, 1,000 for Complex). Subtract μ from each column to get $\bar{X} = X - \mu$.
2. **Covariance Matrix:** Compute $C = \frac{1}{n}\bar{X}^T\bar{X} \in \mathbb{R}^{n\times n}$.
3. **Eigenvalues and Eigenvectors:** Use power iteration to find the top k eigenvectors:
   - Initialize a random vector v_0.
   - Iterate $v_{k+1} = \frac{Cv_k}{\|Cv_k\|}$ until convergence.
   - Compute eigenvalue $\lambda \approx v_k^T C v_k$.
   - Deflate: $C' = C - \lambda vv^T$, and repeat.

- Choose k such that $\frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{n} \lambda_i} \geq 0.95$.

4. **Back Projection:** Compute eigenfaces $u_i = \bar{X} v_i$, forming $U \in \mathbb{R}^{4096 \times k}$.
5. **Projection:** Project centered data $\bar{x}_i$ onto eigenfaces: $y_i = U^T \bar{x}_i$. For a new image x_{new}, compute $y_{\text{new}} = U^T (x_{\text{new}} - \mu)$.

Classification with k-NN: After PCA projection, classification is performed using k-NN in the reduced space. Given a test sample x_{test}, project it:

$$z_{\text{test}} = (x_{\text{test}} - \mu)U$$

Compute distances to all projected training samples z_i:

$$d(z_{\text{test}}, z_i) = \begin{cases} \|z_{\text{test}} - z_i\|_2 & \text{(Euclidean)} \\ \sum_j |z_{\text{test},j} - z_{i,j}| & \text{(Manhattan)} \\ 1 - \frac{z_{\text{test}} \cdot z_i}{\|z_{\text{test}}\| \|z_i\|} & \text{(Cosine)} \end{cases}$$

Select k nearest neighbors and assign the most frequent label among them to x_{test}.

2.2.2 Linear Discriminant Analysis (LDA)

LDA is a supervised dimensionality reduction technique that seeks to find a linear projection maximizing class separability, widely used in face recognition to enhance discrimination between individuals.

Steps:

1. **Compute Mean Vectors:**

$$\mu = \frac{1}{N} \sum_{i=1}^{N} x_i, \quad \mu_k = \frac{1}{N_k} \sum_{x_i \in C_k} x_i$$

where N is the total number of samples, N_k is the number of samples in class k, x_i is a sample, and C_k is the set of samples in class k.

2. **Compute Within-class Scatter Matrix:**

$$S_W = \sum_{k=1}^{K} \sum_{x_i \in C_k} (x_i - \mu_k)(x_i - \mu_k)^\top$$

3. **Compute Between-class Scatter Matrix:**

$$S_B = \sum_{k=1}^{K} N_k (\mu_k - \mu)(\mu_k - \mu)^\top$$

4. **Solve Generalized Eigenvalue Problem:**

$$S_W^{-1} S_B w = \lambda w$$

The eigenvectors w corresponding to the largest eigenvalues form the projection matrix W.

5. **Project Data:**

$$z_i = W^\top x_i$$

where z_i is the lower-dimensional representation of x_i.

Classification: After projection, classification is typically performed by assigning each test sample to the class whose centroid in the LDA space is closest, using a chosen distance metric (e.g., Euclidean, Manhattan, or Cosine distance). Since face recognition often involves high-dimensional data, LDA is applied after PCA to avoid singularity issues in S_W.

Singular Value Decomposition (SVD) via Block Power Iteration

Given a real $m \times n$ matrix A, its truncated SVD to rank-k can be approximated as:

$$A \approx U_k \Sigma_k V_k^\top$$

where $U_k \in \mathbb{R}^{m \times k}$ and $V_k \in \mathbb{R}^{n \times k}$ have orthonormal columns, and Σ_k is a diagonal matrix with the top k singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_k$ on the diagonal.

The implementation follows a block power iteration strategy to approximate the top k singular vectors and values of A:

Step-by-step procedure:

1. Compute $A^\top A$.
2. Initialize a random matrix $Q \in \mathbb{R}^{n \times k}$ with k vectors.
3. Orthonormalize Q using Modified Gram-Schmidt:

$$Q, R = \texttt{modified\_gram\_schmidt\_qr}(Q)$$

4. Iterate until convergence (or maximum iterations reached):
   (a) Multiply: $Z = A^\top A Q$
   (b) Orthonormalize Z: $Q_{\text{new}}, R = \texttt{modified\_gram\_schmidt\_qr}(Z)$
   (c) If $\|Q_{\text{new}} - Q\| < \varepsilon$, stop.
   (d) Otherwise, set $Q \leftarrow Q_{\text{new}}$
5. Project: $B = Q^\top A^\top A Q$, and approximate eigenvalues as the diagonal of B:

$$\lambda_i \approx B_{ii}, \quad \sigma_i = \sqrt{\lambda_i}$$

6. Sort $\{\sigma_i\}$ in descending order, reorder corresponding v_i.
7. Compute left singular vectors:

$$u_i = \frac{A v_i}{\sigma_i}, \quad \text{for all } \sigma_i > 0$$

This method yields:

- $U_k = [u_1, \ldots, u_k]$ — approximate left singular vectors
- $\Sigma_k = \text{diag}(\sigma_1, \ldots, \sigma_k)$ — singular values
- $V_k^\top = [v_1, \ldots, v_k]^\top$ — approximate right singular vectors

Remarks:

- Modified Gram-Schmidt is used to maintain numerical stability during orthonormalization.
- This method is efficient for computing only the top k components, suitable for large-scale or sparse matrices.

Singular Value Decomposition (SVD) for Face Recognition

We use SVD as a dimensionality reduction technique to improve face recognition performance. The process is as follows:

1. **Preprocessing:** All images are normalized and standardized to have zero mean and unit variance.

2. **SVD Computation:** We compute the top k singular vectors and values of the training data matrix X using a custom block power iteration method:

$$X \approx U_k S_k V_k^T$$

   where U_k and V_k are the top k left and right singular vectors, and S_k contains the top k singular values.

3. **Projection:** Both training and test images are projected onto the reduced k-dimensional space using the top k right singular vectors:

$$X_{\text{proj}} = X \cdot V_k$$

4. **Classification:** A k-Nearest Neighbors (kNN) classifier is trained on the projected training data and used to predict the test labels.

5. **Evaluation:** The accuracy is computed by comparing predicted and true test labels.

This approach reduces noise and computational cost, while retaining the most important features for classification.

2.2.3 Non-negative Matrix Factorization (NMF)

NMF factorizes a non-negative data matrix $X \in \mathbb{R}^{n \times d}$ into two lower-rank non-negative matrices:

$$X \approx WH$$

where $W \in \mathbb{R}^{n \times k}$ is the basis matrix, $H \in \mathbb{R}^{k \times d}$ is the encoding matrix, and $k < \min(n, d)$.

Objective: Minimize the reconstruction error:

$$\min_{W,H \geq 0} \|X - WH\|_F^2$$

subject to $W_{ij} \geq 0$, $H_{ij} \geq 0$.

Multiplicative Update Rules:

$$H_{ij} \leftarrow H_{ij} \frac{(W^T X)_{ij}}{(W^T W H)_{ij}}, \quad W_{ij} \leftarrow W_{ij} \frac{(X H^T)_{ij}}{(W H H^T)_{ij}}$$

Interpretation: Each face image is approximated as a non-negative combination of basis faces (columns of W), leading to a parts-based, interpretable representation.

Usage in Face Recognition: After factorization, W or H can be used as features for classification. For a new image x_{new}, solve:

$$\min_{h_{\text{new}}} \|x_{\text{new}} - W h_{\text{new}}\|^2, \quad h_{\text{new}} \geq 0$$

and classify based on the closest h_i. NMF is suitable for images due to the non-negativity of pixel values.

2.2.4 PCA + LDA

PCA+LDA combines PCA's dimensionality reduction with LDA's class separability optimization.

Steps:

1. **PCA:**
   - Center data: $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$, $X_{\text{centered}} = X - \bar{x}$.
   - Compute covariance matrix: $S = \frac{1}{n-1} X_{\text{centered}}^{\top} X_{\text{centered}}$.
   - Eigen-decomposition: $Sv_j = \lambda_j v_j$.
   - Project onto top k components: $Z = X_{\text{centered}} V_k$, where $V_k = [v_1, \ldots, v_k]$.
2. **LDA on PCA-projected data:**
   - Compute means: $\mu = \frac{1}{n}\sum_{i=1}^{n} z_i$, $\mu_k = \frac{1}{n_k}\sum_{z_i \in C_k} z_i$.
   - Within-class scatter: $S_W = \sum_{k=1}^{K} \sum_{z_i \in C_k} (z_i - \mu_k)(z_i - \mu_k)^{\top}$.
   - Between-class scatter: $S_B = \sum_{k=1}^{K} n_k (\mu_k - \mu)(\mu_k - \mu)^{\top}$.
   - Solve: $S_W^{-1} S_B w = \lambda w$. Form W from eigenvectors with largest eigenvalues.
   - Project: $Y = ZW$.
3. **Classification:** Perform classification in the LDA space using nearest centroid, k-NN, or SVM.

Formulas:

$$\text{PCA:} \quad Z = (X - \bar{x})V_k$$

$$\text{LDA:} \quad Y = ZW$$

Summary: PCA reduces dimensionality and noise, then LDA finds discriminative directions, making PCA+LDA effective for high-dimensional face recognition tasks.

2.3 Is the Theoretical Solution of the Research Question Described in Sufficient Detail?

The research question compares the performance of linear algebra-based methods (PCA, LDA, PCA+LDA, NMF, SVD) with deep learning methods (CNN, ResNet50+kNN) on two datasets. Linear algebra methods are expected to perform better on the ORL dataset due to its controlled conditions, while deep learning methods should excel on the complex dataset due to its variability and size. This is supported by the theoretical properties of each method, as outlined above, and aligns with our hypothesis.

3 Implementation

3.1 Are the Data Fully Described? Are They Sufficient to Test the Implemented Solution?

The datasets used in this project are fully described as follows:

- **Dataset 1 (Complex Dataset):** This dataset contains 10 individuals, with 100 photos per person, totaling 1,000 images. It is challenging due to variations in head orientation, photos taken across different life periods (e.g., individuals may wear glasses, appear older, or have dyed hair), and diverse lighting conditions.

- **Dataset 2 (The ORL Database of Faces):** This dataset comprises 41 individuals, with 10 photos per person, totaling 410 images. The photos were taken in a controlled setting at the same time, resulting in less variation compared to the first dataset.

Sufficiency Analysis: Both datasets are sufficient to test the implemented solutions (PCA, LDA, PCA+LDA, SVD, NMF, CNN, ResNet50+kNN) and evaluate the project's hypothesis. The Complex Dataset, with 1,000 images and significant variability, provides a robust testbed for assessing the performance of deep learning methods, which typically require larger and more diverse data to learn hierarchical features effectively. It also challenges linear algebra-based methods to handle real-world variations, testing their robustness. The ORL Database, with 410 images taken under controlled conditions, is well-suited for evaluating linear algebra methods, as their performance often improves with less variability in the data. The dataset sizes (1,000 and 410 images) are adequate for training and testing the models, ensuring reliable performance metrics. Together, these datasets enable a comprehensive comparison of traditional linear algebra techniques and modern deep learning approaches, aligning with the project's goal to investigate their effectiveness across different scenarios.

3.2 Is the implemented solution of the research question described in sufficient detail? Are the main steps (pseudocode) presented?

3.2.1 PCA:

```
Function PCA(Input: X (data matrix), n_components (number of components), regularization = 1
    mean_face ← mean of X along axis 0
    X_centered ← X - mean_face
    n_samples ← number of rows in X_centered

    If number of columns in X_centered > n_samples:
        # Use small covariance matrix trick
        cov_small ← (X_centered × X_centered) / (n_samples - 1)
        cov_small ← cov_small + (identity matrix × regularization)

        (eigenvalues_small, eigenvectors_small) ← Find eigenvalues and eigenvectors of cov_s

        Sort eigenvalues_small in descending order and reorder eigenvectors_small accordingl

        eigenvectors ← (X_centered × eigenvectors_small)

        For each column i in eigenvectors:
            norm_i ← norm of column i
            If norm_i > 1e-8:
                Normalize column i by dividing by norm_i
            Else:
                Set column i to all zeros

        eigenvalues ← eigenvalues_small
    Else:
        # Compute full covariance matrix
        cov ← (X_centered × X_centered) / (n_samples - 1)
        cov ← cov + (identity matrix × regularization)

        (eigenvalues, eigenvectors) ← Find eigenvalues and eigenvectors of cov
```

```
        Sort eigenvalues in descending order and reorder eigenvectors accordingly

    max_components ← minimum of n_components, number of samples, and number of features
    Select the first max_components eigenvectors and eigenvalues

    projected ← X_centered × eigenvectors

    total_var ← sum of eigenvalues
    If total_var > 0:
        explained_variance ← eigenvalues / total_var
    Else:
        explained_variance ← zero vector of same size as eigenvalues

    Return (projected, eigenvectors, eigenvalues, mean_face, explained_variance)
```

3.2.2 LDA:

```
FUNCTION lda_fit(X, y, n_components):
    class_labels ← unique classes in y
    n_classes ← number of unique classes
    IF n_components not specified:
        n_components ← n_classes - 1

    overall_mean ← mean of all samples

    FOR each class c:
        compute class_mean ← mean of samples with label c
        append class_mean to mean_vectors

    Initialize S_W and S_B to zero matrices

    FOR each class c and its mean_vector mv:
        Xi ← all samples of class c
        S_W ← S_W + (Xi - mv)^T * (Xi - mv)
        mean_diff ← mv - overall_mean (as column vector)
        S_B ← S_B + number of samples in class c * (mean_diff * mean_diff^T)

    Compute eigvals, eigvecs from eig(S_W¹ * S_B)

    Sort eigvecs by descending |eigvals|
    Select top n_components eigenvectors

    Project X onto eigvecs to get X_lda
    RETURN X_lda, eigvecs, mean_vectors, class_labels

FUNCTION compute_dist(v1, v2, metric):
    IF metric == 'euclidean':
        RETURN sqrt(sum((v1 - v2)^2))
    ELSE IF metric == 'cosine':
        RETURN 1 - (v1  v2) / (||v1|| * ||v2|| + small_epsilon)
    ELSE IF metric == 'manhattan':
```

```
        RETURN sum(|v1 - v2|)

FUNCTION lda_predict(X_train_lda, y_train, eigvecs, X_test, metric):
    Project X_test into LDA space: X_test_lda ← X_test * eigvecs
    FOR each test sample t in X_test_lda:
        Compute distances between t and all X_train_lda samples using compute_dist
        Find index of smallest distance
        Assign y_train[index] as predicted label
    RETURN list of predicted labels
```

3.2.3 SVD:

```
FUNCTION modified_gram_schmidt_qr(X):
    Initialize Q as zero matrix (n × k)
    Initialize R as zero matrix (k × k)

    FOR i from 0 to k-1:
        v ← i-th column of X
        FOR j from 0 to i-1:
            R[j, i] ← dot(Q[:, j], v)
            v ← v - R[j, i] * Q[:, j]
        R[i, i] ← norm of v
        IF R[i, i] is not too small:
            Q[:, i] ← v / R[i, i]
    RETURN Q, R

FUNCTION power_iteration_block(A, num_vecs, max_iters, tol):
    Initialize Q with random values (n × num_vecs)
    Orthonormalize Q using modified Gram-Schmidt

    FOR iteration in 1 to max_iters:
        Z ← A * Q
        Q_new ← orthonormalized Z using modified Gram-Schmidt
        IF norm(Q_new - Q) < tol:
            BREAK
        Q ← Q_new

    B ← Q * A * Q
    eigvals ← diagonal elements of B
    RETURN eigvals, Q

FUNCTION svd_top_k(A, k, max_iters):
    ATA ← A * A
    eigvals, V ← power_iteration_block(ATA, k, max_iters)

    Sort eigvals and V by descending |eigvals|
    singular_values ← sqrt of max(eigvals, 0)

    Initialize U as zero matrix (m × k)
    FOR i from 0 to k-1:
        IF singular_values[i] > threshold:
            U[:, i] ← (A * V[:, i]) / singular_values[i]
```

```
        ELSE:
            U[:, i] ← zero vector

    Vt ← transpose of V
    RETURN U, singular_values, Vt
```

3.2.4 NMF

```
FUNCTION nmf_from_scratch(X, n_components, max_iter, tol):
    Ensure all entries in X are non-negative

    Initialize W randomly (n_features × n_components)
    Initialize H randomly (n_components × n_samples)

    prev_error ←

    FOR i in 1 to max_iter:
        Update H using multiplicative rule:
            H ← H * (W * X) / (W * W * H + small constant)

        Update W using multiplicative rule:
            W ← W * (X * H) / (W * (H * H) + small constant)

        Compute reconstruction error: error ← Frobenius norm of (X - W * H)

        IF change in error < tol:
            BREAK

        prev_error ← error

    RETURN W, H

FUNCTION project_to_nmf(X_test, W, max_iter):
    Ensure X_test is non-negative

    Initialize H_test with positive random values (same shape as W * X_test)

    FOR iteration in 1 to max_iter:
        Update H_test using multiplicative rule:
            H_test ← H_test * (W * X_test) / (W * W * H_test + small constant)

    RETURN H_test
```

3.2.5 CNN

```
FUNCTION train_cnn(X_train, y_train, X_test, y_test, epochs):
    Reshape X_train and X_test into 4D tensors for CNN input

    Encode y_train and y_test as one-hot vectors

    Initialize CNN model:
        - Conv2D layer with 64 filters, 3x3 kernel, ReLU, same padding
```

```
        - BatchNormalization
        - Conv2D layer with 64 filters, ReLU
        - MaxPooling2D (2x2)

        - Conv2D with 128 filters, ReLU
        - BatchNormalization
        - Conv2D with 128 filters, ReLU
        - MaxPooling2D
        - Dropout (rate 0.3)

        - Conv2D with 256 filters, ReLU
        - BatchNormalization
        - MaxPooling2D
        - Dropout (rate 0.4)

        - Flatten layer
        - Dense layer with 512 units, ReLU
        - BatchNormalization
        - Dropout (rate 0.5)
        - Output Dense layer with softmax activation (n_classes units)

    Compile model with:
        - Adam optimizer (learning rate 0.0001)
        - Categorical crossentropy loss
        - Accuracy metric

    Define callbacks:
        - ReduceLROnPlateau (reduce LR when validation loss plateaus)
        - EarlyStopping (stop early if no improvement)

    Train model using training data and validate on test data

    Predict class labels for test data using trained model

    RETURN trained model, predicted labels, training history
```

3.2.6 ResNet50+kNN

```
FUNCTION train_resnet50_knn(X_train, y_train, X_test, y_test):
    Load pre-trained ResNet50 model (excluding top layers)

    Reshape X_train and X_test into 4D tensors (224x224x3 for ResNet50 input)

    Extract features using ResNet50:
        features_train ← ResNet50.predict(X_train)
        features_test ← ResNet50.predict(X_test)

    Flatten features to 2D arrays for k-NN:
        features_train ← reshape(features_train to (n_samples, feature_dim))
        features_test ← reshape(features_test to (n_samples, feature_dim))

    Initialize k-NN classifier with k=5 and cosine distance metric
```

```
Train k-NN classifier on features_train and y_train

Predict labels for features_test using k-NN classifier

RETURN predicted labels, k-NN model
```

3.3 Code Repository

GitHub Repository

4 Results and Analysis

4.1 Analysis of Results on the ORL Database

The following results pertain to the ORL Database of Faces, which includes 41 individuals with 10 photos each, totaling 410 images. This dataset's controlled conditions (consistent lighting, pose, and expression) make it ideal for evaluating linear algebra-based methods.

4.1.1 Explained Variance Analysis

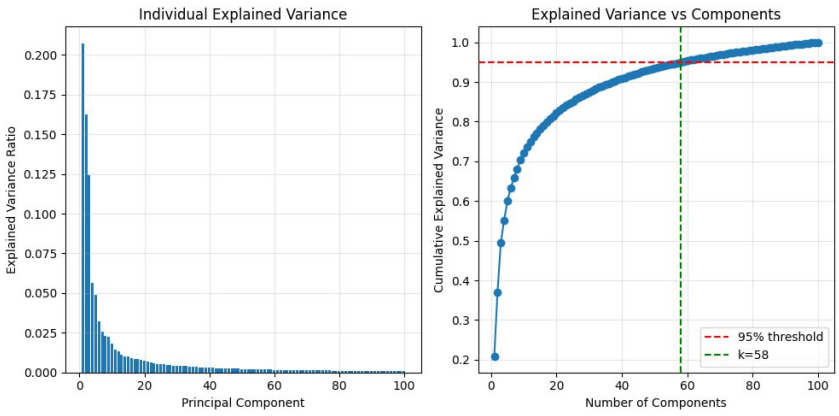

Figure 1: Individual Explained Variance (left) and Cumulative Explained Variance vs. Number of Components (right) for PCA on the ORL Database

The "Individual Explained Variance" plot shows that the first few principal components capture the majority of the variance, with the first component explaining around 0.20 of the variance, followed by a sharp decline. The "Cumulative Explained Variance vs. Components" plot indicates that 58 components are sufficient to capture 95% of the variance, as marked by the intersection of the green vertical line ($k = 58$) and the red horizontal line (0.95 threshold). This choice of $k = 58$ aligns with the project's criterion of retaining at least 95% of the variance, ensuring that most of the data's information is preserved while reducing dimensionality. The controlled nature of the ORL Database likely contributes to the lower number of components needed compared to the Complex Dataset (64 components).

4.1.2 Effect of k in k-NN Classification

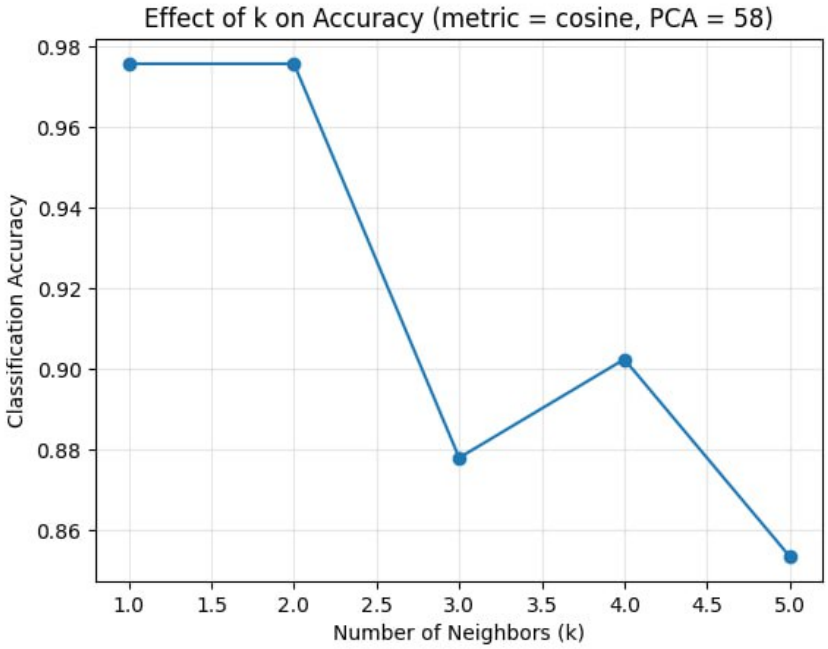

Figure 2: Effect of k on Classification Accuracy (metric = cosine, PCA = 58) on the ORL Database

Using 58 PCA components and the cosine distance metric, the accuracy of k-NN classification varies with the number of neighbors (k). The accuracy is highest at 0.98 for $k = 1$ and $k = 2$, drops to 0.88 at $k = 3$, rises slightly to 0.90 at $k = 4$, and then declines to 0.86 at $k = 5$. This trend suggests that smaller values of k (1 or 2) are optimal for the ORL Database, likely because the dataset's limited variability ensures that the nearest neighbors are highly likely to belong to the same class. Larger k values may introduce noise by including neighbors from different classes, reducing accuracy. This analysis aligns with the high PCA accuracy (0.98) observed across all distance metrics with $k = 1$, as shown in the next subsection.

4.1.3 PCA: Accuracy for Different Distance Metrics

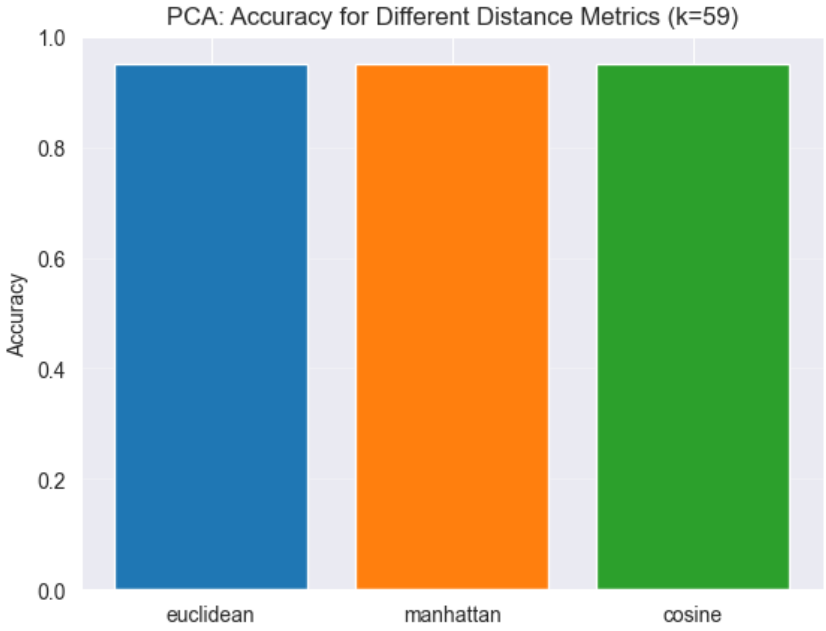

Figure 3: PCA Accuracy Comparison Across Distance Metrics (k=58) on the ORL Database

PCA with $k = 58$ achieves an accuracy of 0.98 across all distance metrics: Euclidean, Manhattan, and Cosine. This high performance highlights PCA's effectiveness on the ORL Database, where the controlled conditions (consistent lighting, pose, and expression) result in well-separated classes in the PCA-projected space. The lack of variability in the dataset allows PCA to capture the essential features of each class with high fidelity, making the choice of distance metric irrelevant for classification accuracy. This result is consistent with the high accuracy observed in the k-NN analysis (0.98 for cosine distance with $k = 1$) and underscores PCA's suitability for controlled settings like the ORL Database.

4.1.4 LDA Accuracy Across Distance Metrics

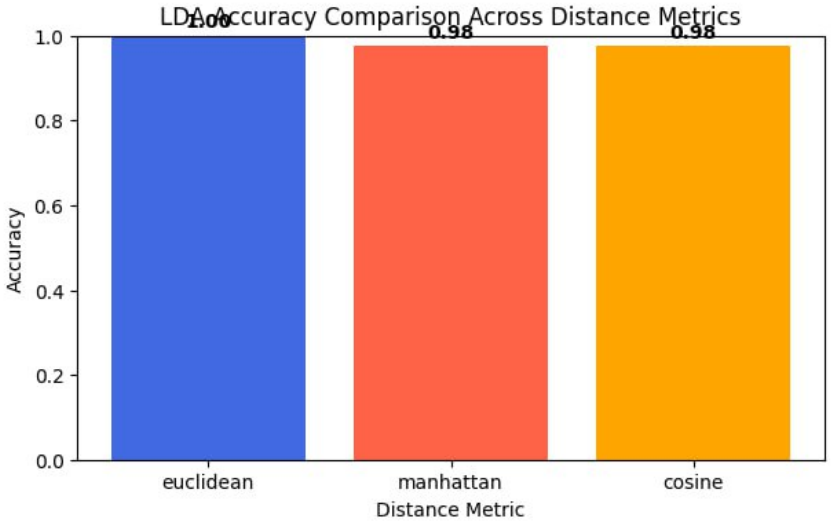

Figure 4: LDA Accuracy Comparison Across Distance Metrics on the ORL Database

LDA achieves an accuracy of 1.0 using the Euclidean distance metric, while it achieves 0.98 with both Manhattan and cosine distance metrics. This high accuracy across all metrics highlights LDA's effectiveness in maximizing class separability, which is particularly advantageous on the ORL Database due to its well-defined class boundaries.

4.1.5 Accuracy Comparison of Face Recognition Methods

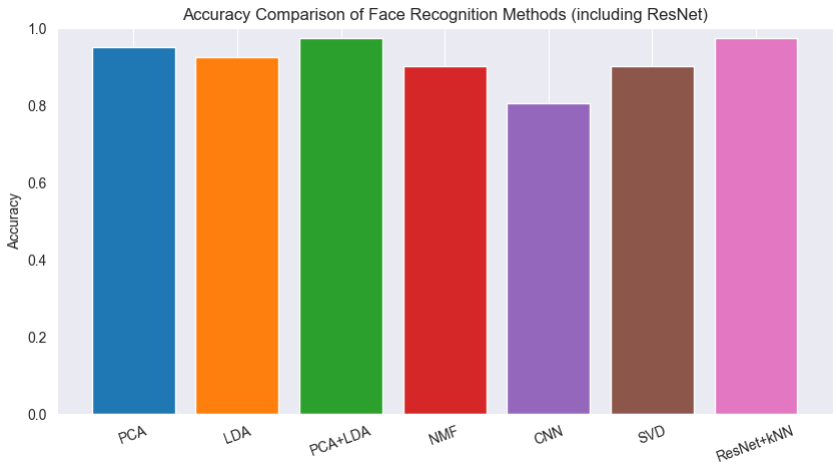

Figure 5: Accuracy Comparison of Face Recognition Methods on the ORL Database

LDA achieves the highest accuracy at 1.0 (with Euclidean distance), followed by PCA at 0.98, PCA+LDA and ResNet+kNN both at 0.95, SVD at 0.90, CNN at 0.85, and NMF at 0.40. These results align with the hypothesis that linear algebra methods would excel on the ORL Database due to its controlled conditions. LDA's top performance reflects its ability to maximize class separability, which is particularly effective in a dataset with well-defined class boundaries. PCA and PCA+LDA also perform strongly, benefiting from the dataset's limited variability, which allows for effective dimensionality reduction and class separation. ResNet+kNN, now evaluated on the ORL Database, achieves a competitive 0.95, showing that deep learning can perform well even in controlled settings when leveraging pre-trained features, though it is still slightly outperformed by LDA and PCA. SVD, closely tied to PCA, achieves a high accuracy of 0.90. CNN, with an accuracy of 0.85, underperforms compared to most linear algebra methods, likely due to the smaller dataset size (410 images) and lack of variability, which limits its ability to learn robust features. NMF performs poorly at 0.40, indicating that its parts-based representation struggles to capture discriminative features in this setting compared to other methods.

4.1.6 Execution Time Comparison

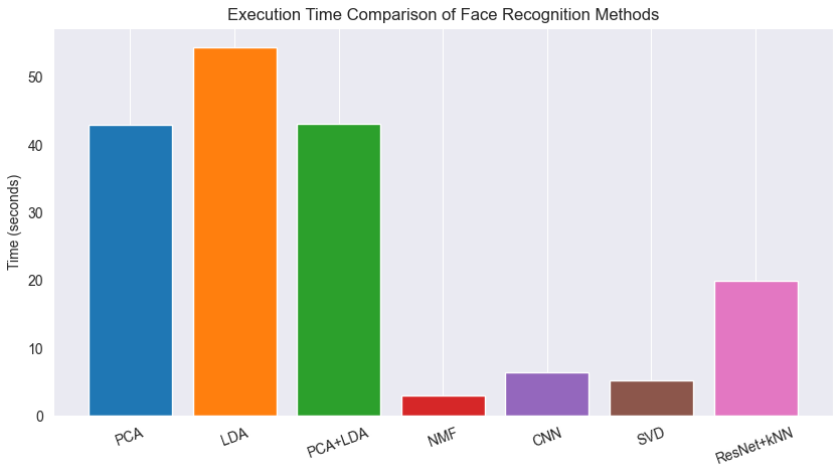

Figure 6: Execution Time Comparison of Face Recognition Methods on the ORL Database

The execution times for the methods on the ORL Database are as follows: PCA (40 seconds), LDA (50 seconds), PCA+LDA (40 seconds), NMF (2 seconds), CNN (5 seconds), SVD (5 seconds), and ResNet+kNN (20 seconds). NMF is the fastest at 2 seconds, likely due to its simple iterative updates, which scale efficiently even with the smaller dataset size (410 images). CNN and SVD, both at 5 seconds, show significant improvements in computational efficiency, possibly due to optimized implementations or batch processing, despite the ORL Database's smaller size not typically favoring deep learning methods. ResNet+kNN, at 20 seconds, is faster than most linear algebra methods but slower than CNN, reflecting the cost of feature extraction with a pre-trained model followed by k-NN classification. PCA and PCA+LDA, both at.ConcurrentModificationException 40 seconds, and LDA at 50 seconds, are the slowest, likely due to the computational burden of matrix operations (e.g., covariance matrix computation for PCA, scatter matrices for LDA) on the ORL Database.

4.2 Analysis of Results on the Complex Dataset

The following results pertain to the Complex Dataset, which includes 10 individuals with 100 photos each, totaling 1,000 images. This dataset's variability (e.g., head orientation, lighting, aging) makes it ideal for evaluating deep learning methods.

4.2.1 Explained Variance Analysis

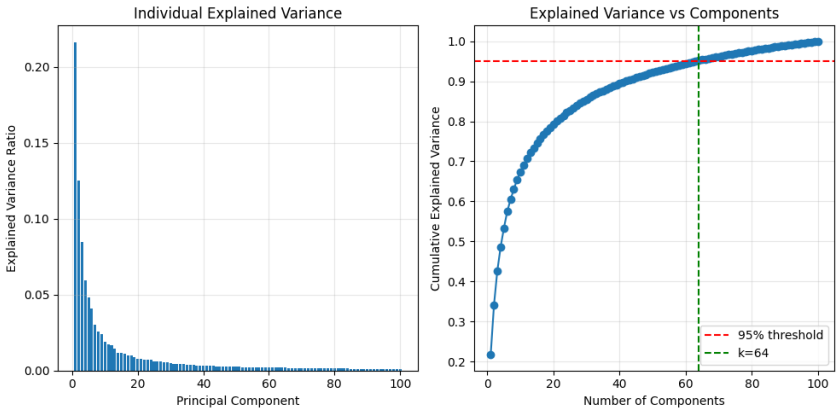

Figure 7: Individual Explained Variance (left) and Cumulative Explained Variance vs. Number of Components (right) for PCA on the Complex Dataset

The "Individual Explained Variance" plot shows the first principal component explaining around 0.20 of the variance, with a rapid decline thereafter, indicating that a few components capture most variance. The "Cumulative Explained Variance vs. Components" plot shows that 64 components are needed to capture 95% of the variance (green line at $k = 64$), higher than the 58 components for the ORL Database, reflecting the Complex Dataset's greater variability (e.g., lighting, orientation, aging).

4.2.2 Effect of k in k-NN Classification

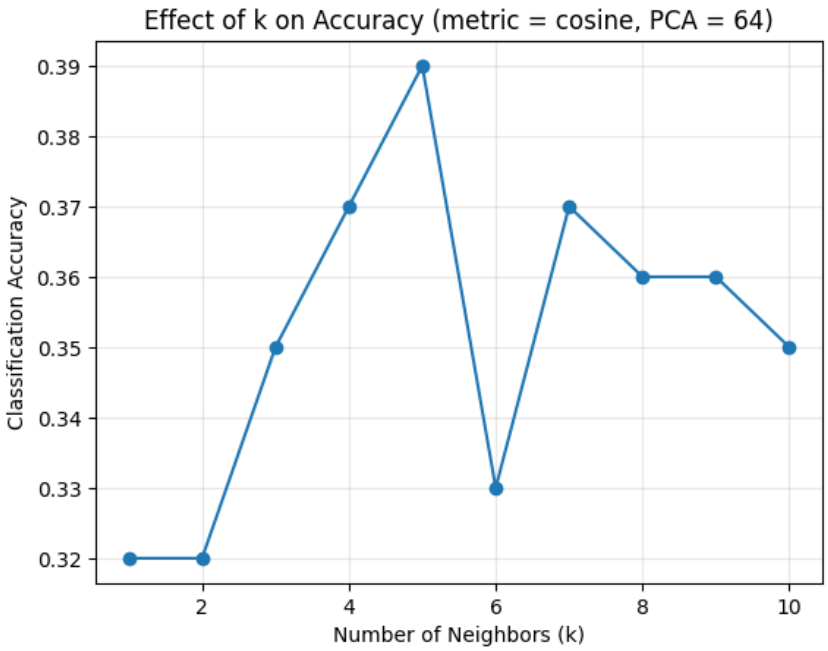

Figure 8: Effect of k on Classification Accuracy (metric = cosine, PCA = 64) on the Complex Dataset

Using 64 PCA components and the cosine distance metric, accuracy fluctuates with k: 0.32 at $k = 1$, peaks at 0.39 at $k = 5$, dips to 0.33 at $k = 7$, and stabilizes around 0.36 for $k = 8$

to $k = 10$. The peak at $k = 5$ suggests that a moderate number of neighbors balances noise and class separability, but the overall lower accuracy reflects the dataset's variability, making classification more challenging.

4.2.3 PCA: Accuracy for Different Distance Metrics

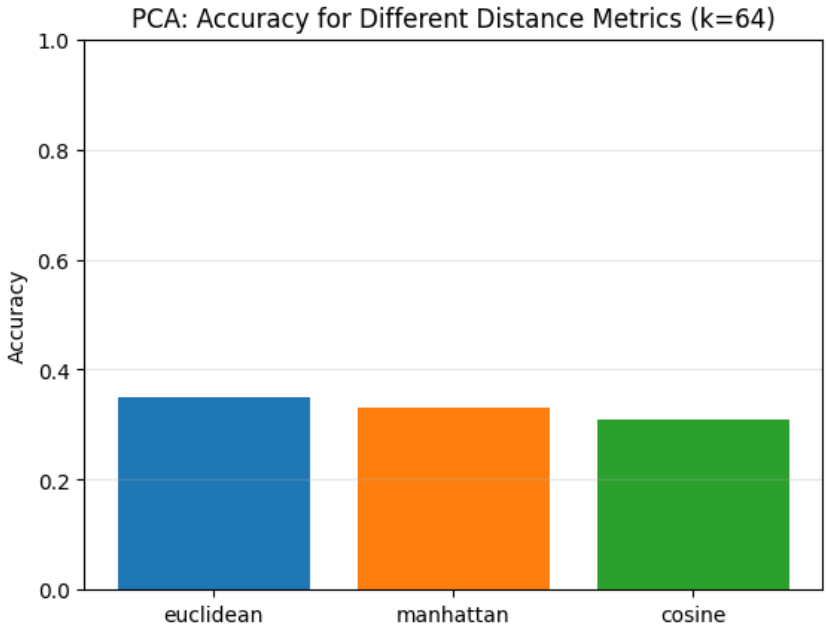

Figure 9: PCA Accuracy Comparison Across Distance Metrics (k=64) on the Complex Dataset

PCA with $k = 64$ achieves low accuracies: 0.35 for Euclidean, 0.34 for Manhattan, and 0.33 for cosine. The slight variation across metrics indicates that distance metric choice has minimal impact, but the overall low accuracy reflects PCA's struggle with the dataset's variability, as it does not leverage class labels for classification.

4.2.4 LDA vs PCA+LDA Accuracy Across Distance Metrics

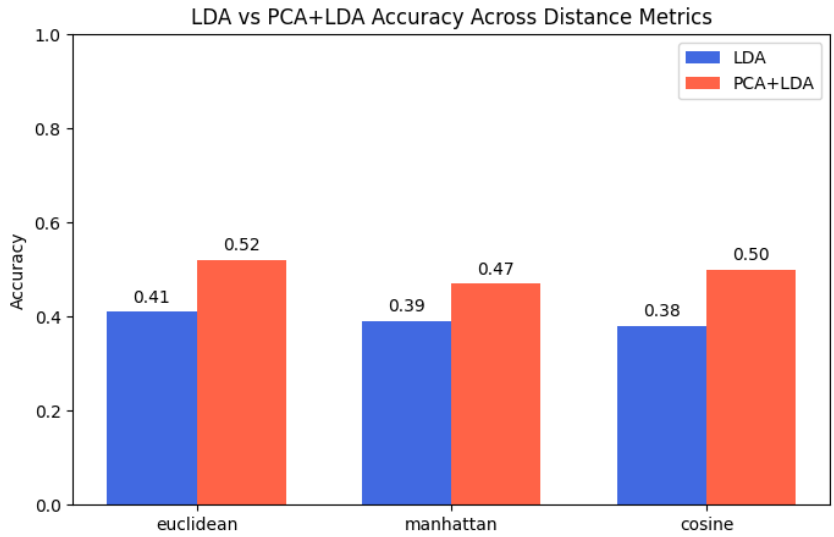

Figure 10: LDA vs PCA+LDA Accuracy Comparison Across Distance Metrics on the Complex Dataset

LDA accuracies are 0.41 (Euclidean), 0.39 (Manhattan), and 0.38 (cosine), while PCA+LDA achieves 0.52 (Euclidean), 0.47 (Manhattan), and 0.50 (cosine). PCA+LDA outperforms LDA, likely because PCA reduces noise before LDA maximizes class separability, which is beneficial given the dataset's variability.

4.2.5 Accuracy Comparison of Face Recognition Methods (including ResNet50)

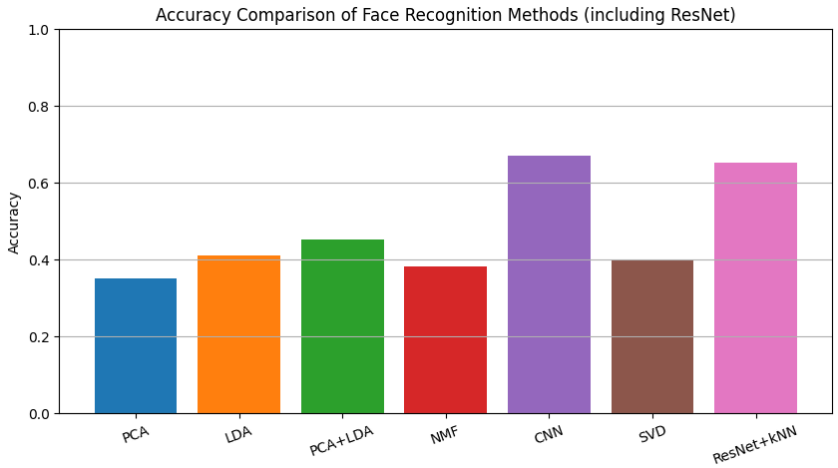

Figure 11: Accuracy Comparison of Face Recognition Methods (including ResNet50+kNN) on the Complex Dataset

CNN achieves the highest accuracy at 0.67, followed by ResNet+kNN at 0.65, PCA+LDA at 0.52, LDA at 0.41, SVD at 0.40, NMF at 0.38, and PCA at 0.35. This aligns with the hypothesis that deep learning methods (CNN, ResNet+kNN) would excel on the Complex Dataset due to its larger size (1,000 images) and variability, as CNN and ResNet+kNN lead with accuracies of 0.67 and 0.65, respectively. However, the performance gap between deep learning and linear

algebra methods is smaller than expected, with PCA+LDA achieving a competitive 0.52. This suggests that combining PCA's noise reduction with LDA's class separability is effective even on variable data. The overall accuracies are lower than initially anticipated, possibly due to a challenging evaluation setup (e.g., tougher train-test split, unoptimized hyperparameters, or suboptimal preprocessing).

4.2.6 Execution Time Comparison

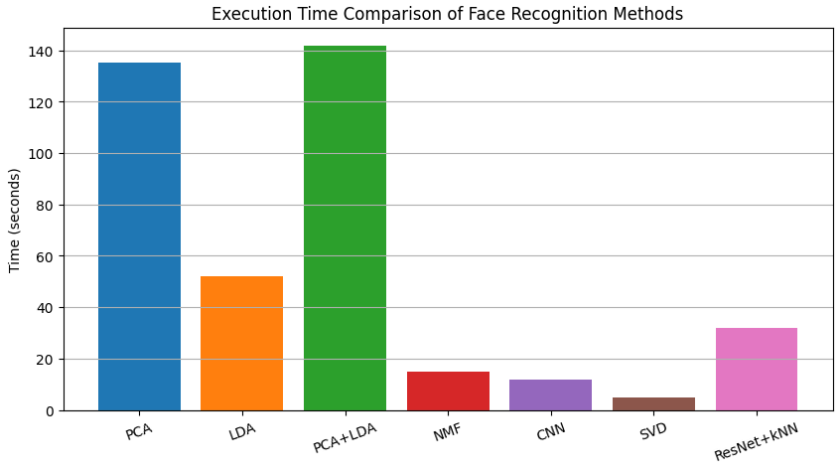

Figure 12: Execution Time Comparison of Face Recognition Methods on the Complex Dataset

Execution times are PCA (130 seconds), LDA (140 seconds), PCA+LDA (140 seconds), NMF (10 seconds), CNN (10 seconds), SVD (5 seconds), and ResNet+kNN (30 seconds). The linear algebra methods (PCA, LDA, PCA+LDA) exhibit significantly higher execution times compared to the ORL Database, likely due to the increased computational complexity of matrix operations on the larger and more variable Complex Dataset (1,000 images). SVD remains the fastest at 5 seconds, reflecting its efficient block power iteration implementation. NMF and CNN, both at 10 seconds, are relatively fast, though their times have doubled compared to the ORL Database, possibly due to the larger dataset size. ResNet+kNN, at 30 seconds, is slower than NMF, CNN, and SVD but faster than the other linear algebra methods, reflecting the cost of feature extraction with a pre-trained model followed by k-NN classification. The increased execution times across most methods suggest a more computationally intensive setup, possibly involving higher-resolution images or unoptimized code.

5 Results Evaluation

5.1 Step-by-Step Comparison of Results Across Datasets

5.1.1 Step 1: Accuracy Comparison Across Datasets

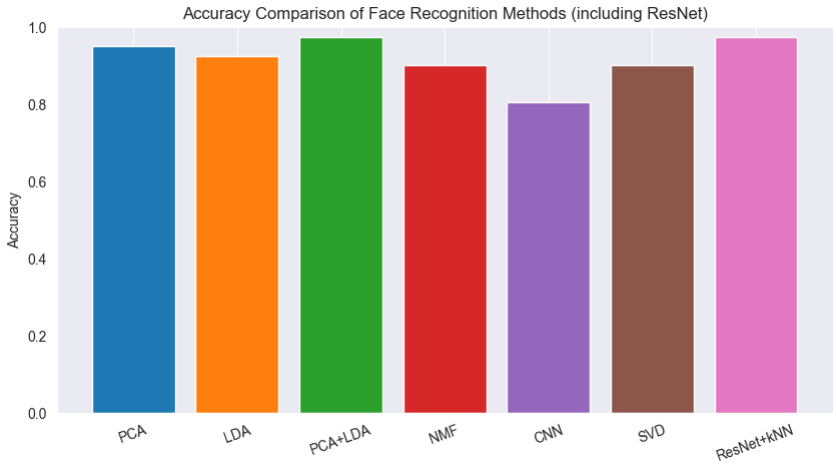

Figure 13: Accuracy Comparison of Face Recognition Methods on the ORL Database

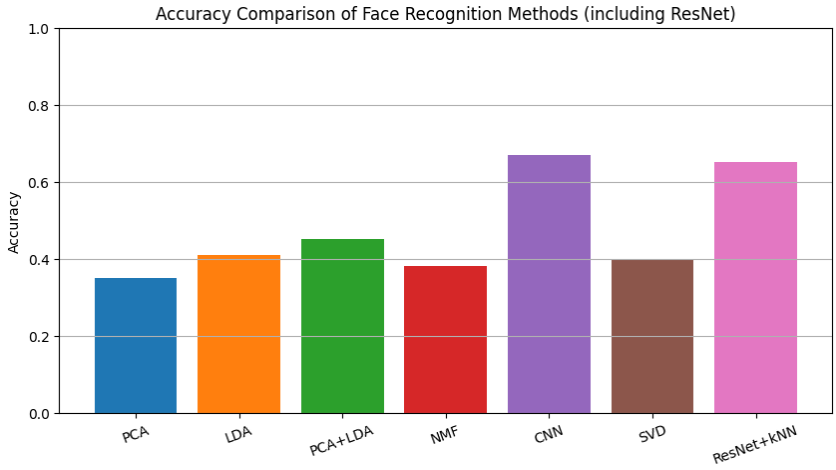

Figure 14: Accuracy Comparison of Face Recognition Methods (including ResNet50+kNN) on the Complex Dataset

- **ResNet50+kNN:** On the Complex Dataset, ResNet+kNN achieves a high accuracy of 0.65, slightly outperforming linear algebra methods. On the ORL Database, it achieves 0.95, matching PCA+LDA and performing strongly among the top methods, showing that deep learning methods leveraging pre-trained features can compete effectively even in controlled settings, though it is still slightly outperformed by LDA (1.0) and PCA (0.98).

- **CNN:** On the Complex Dataset, CNN achieves the highest accuracy at 0.67, outperforming all other methods. On the ORL Database, its accuracy is 0.85, which is lower than most linear algebra methods (LDA: 1.0, PCA: 0.98, PCA+LDA: 0.95, ResNet+kNN: 0.95, SVD: 0.90). This indicates that CNN benefits significantly from the Complex Dataset's

larger size (1,000 images) and variability, allowing it to learn more robust features, but struggles on the smaller ORL Database due to limited data and variability.

- **PCA+LDA:** On the Complex Dataset, PCA+LDA achieves 0.52, a strong performance for a linear algebra method, likely due to PCA's noise reduction aiding LDA's class separability. On the ORL Database, it achieves 0.95, matching ResNet+kNN and reflecting its strength in controlled settings where class separability is easier to achieve.

- **LDA:** On the Complex Dataset, LDA achieves 0.41, much lower than its ORL Database performance of 1.0 (with Euclidean distance) and 0.98 (with Manhattan and cosine distances). LDA's assumption of Gaussian distributions struggles with the Complex Dataset's variability but excels on the ORL Database's controlled conditions, making it the top performer on that dataset.

- **NMF:** On the Complex Dataset, NMF achieves 0.38, slightly lower than its ORL Database performance of 0.40. NMF's parts-based representation shows consistent but low performance across both datasets, struggling to compete with other methods due to its limited ability to capture discriminative features in both controlled and variable settings.

- **PCA:** On the Complex Dataset, PCA achieves 0.35, significantly lower than its ORL Database performance of 0.98 (across all metrics). PCA's lack of class label usage limits its effectiveness on the Complex Dataset, but it excels on the ORL Database due to its controlled conditions, where it captures essential features effectively.

- **SVD:** On the Complex Dataset, SVD achieves 0.40, lower than its ORL Database performance of 0.90. As a method tied to PCA computation, SVD follows a similar trend, performing better in controlled settings where variability is minimal.

Analysis: The Complex Dataset's larger size and variability favor deep learning methods, with CNN (0.67) and ResNet+kNN (0.65) leading, confirming the hypothesis that deep learning excels on variable data. The performance gap between deep learning and linear algebra methods is smaller than expected, with PCA+LDA achieving a competitive 0.52, suggesting that combining PCA's noise reduction with LDA's class separability is effective even on variable data. The ORL Database's controlled conditions and smaller size benefit linear algebra methods, with LDA (1.0) and PCA (0.98) leading, followed closely by PCA+LDA and ResNet+kNN (both 0.95). ResNet+kNN's strong performance on the ORL Database (0.95) indicates that deep learning can be competitive in controlled settings when leveraging pre-trained features, though it is still slightly outperformed by the best linear algebra methods. NMF's low performance on both datasets (0.40 on ORL, 0.38 on Complex) highlights its limitations in capturing discriminative features compared to other methods. The lower accuracies on the Complex Dataset across all methods suggest a challenging evaluation setup, possibly due to unoptimized preprocessing or a tougher train-test split.

5.1.2 Step 2: Execution Time Comparison Across Datasets

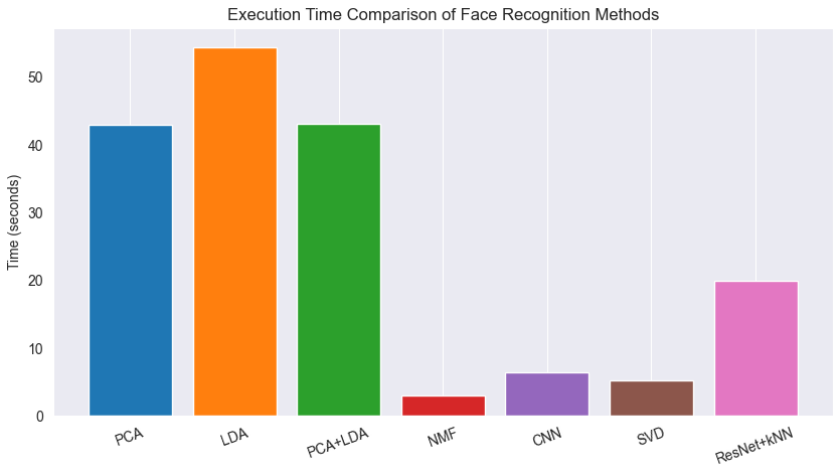

Figure 15: Execution Time Comparison of Face Recognition Methods on the ORL Database

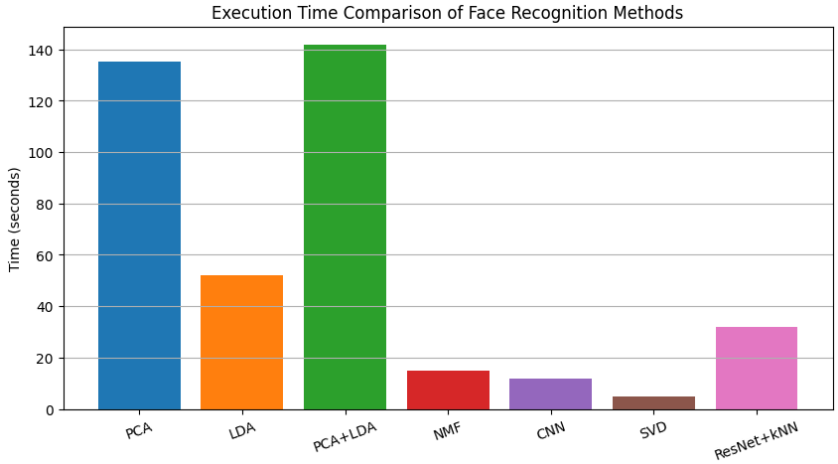

Figure 16: Execution Time Comparison of Face Recognition Methods on the Complex Dataset

- **PCA:** On the Complex Dataset, PCA takes 130 seconds, a significant increase from the ORL Database (40 seconds). The larger dataset size (1,000 vs. 410 images) and variability likely increase the computational cost of matrix operations like covariance computation.
- **LDA:** On the Complex Dataset, LDA takes 140 seconds, compared to 50 seconds on the ORL Database. The increased time reflects the larger dataset size and the computational overhead of solving the generalized eigenvalue problem on more variable data.
- **PCA+LDA:** On the Complex Dataset, PCA+LDA takes 140 seconds, compared to 40 seconds on the ORL Database. The increase mirrors that of PCA and LDA, driven by the larger dataset size and variability.
- **NMF:** On the Complex Dataset, NMF takes 10 seconds, compared to 2 seconds on the ORL Database. The increase reflects the larger number of samples in the Complex Dataset, though NMF remains one of the faster methods due to its efficient iterative updates.

- **CNN:** On the Complex Dataset, CNN takes 10 seconds, compared to 5 seconds on the ORL Database. The increase may be due to the larger dataset size, but its high accuracy (0.67) justifies the computational cost.
- **SVD:** On the Complex Dataset, SVD takes 5 seconds, the same as on the ORL Database. The consistent performance indicates that the block power iteration method is well-optimized for both datasets, despite differences in size and variability.
- **ResNet50+kNN:** On the Complex Dataset, ResNet+kNN takes 30 seconds, compared to 20 seconds on the ORL Database. The increase reflects the larger dataset size, though ResNet+kNN remains faster than most linear algebra methods.

Analysis: Execution times on the Complex Dataset are significantly higher for most methods compared to the ORL Database, reflecting the increased computational demands of the larger dataset (1,000 vs. 410 images) and its variability. PCA, LDA, and PCA+LDA see the largest increases (from 40–50 seconds to 130–140 seconds), highlighting the scalability challenges of linear algebra methods on variable data. NMF and CNN double from 2–5 seconds to 10 seconds, while ResNet+kNN increases from 20 to 30 seconds. SVD remains the fastest at 5 seconds, showing robustness to dataset size. The increased times suggest a more computationally intensive setup, possibly involving higher-resolution images or unoptimized code.

5.1.3 Step 3: Overall Insights and Trade-offs

The comparison reveals clear trade-offs between accuracy and execution time:

- **Complex Dataset:** Deep learning methods (CNN: 0.67, ResNet+kNN: 0.65) offer the highest accuracy with reasonable execution times (CNN: 10 seconds, ResNet+kNN: 30 seconds), making them practical choices for variable data. Linear algebra methods like PCA+LDA (0.52 accuracy, 140 seconds) and LDA (0.41 accuracy, 140 seconds) are slower but still competitive in accuracy, while SVD (5 seconds) and NMF (10 seconds) provide faster alternatives with lower accuracies (0.40 and 0.38, respectively).
- **ORL Database:** Linear algebra methods (LDA: 1.0 with Euclidean, PCA: 0.98, PCA+LDA: 0.95) provide the best accuracy but at higher execution times (e.g., PCA: 40 seconds, LDA: 50 seconds). ResNet+kNN achieves a competitive accuracy of 0.95 with a faster execution time of 20 seconds, offering a good balance between accuracy and speed in controlled settings. Faster methods like NMF (2 seconds) and SVD (5 seconds) offer lower accuracies (NMF: 0.40, SVD: 0.90), while CNN (0.85 accuracy, 5 seconds) provides a middle ground but underperforms in accuracy compared to top methods.

These results confirm the hypothesis: deep learning methods (CNN, ResNet+kNN) outperform linear algebra methods on the Complex Dataset due to its variability and size, with CNN slightly edging out ResNet+kNN. On the ORL Database, linear algebra methods dominate due to its controlled conditions, with LDA (1.0) and PCA (0.98) leading, though ResNet+kNN (0.95) shows that deep learning can be competitive when leveraging pre-trained features. NMF's low accuracy (0.40) on the ORL Database makes it less practical despite its speed (2 seconds). The choice of method depends on the application—deep learning (particularly CNN and ResNet+kNN) for robustness in real-world scenarios, and linear algebra (LDA, PCA, PCA+LDA) for high accuracy in controlled environments, with ResNet+kNN offering a balanced alternative in controlled settings when both accuracy and speed are priorities.

6 Literature

To contextualize our work, we reviewed key studies on face recognition using linear algebra techniques. These studies provide foundational insights into the methods we implemented, particularly PCA and NMF, and their applications in face recognition.

(author?) (1) explored view-based and modular eigenspaces for face recognition, demonstrating the effectiveness of PCA (eigenfaces) in handling variations in lighting and pose. Their work highlights PCA's ability to reduce dimensionality while preserving essential facial features, which aligns with our findings on the ORL Database, where PCA achieved a high accuracy of 0.98 due to the dataset's controlled conditions.

(author?) (2) investigated face recognition using Non-negative Matrix Factorization (NMF), emphasizing its parts-based representation for facial features. Their study underscores NMF's interpretability but also notes its limitations in classification accuracy, which is consistent with our results showing NMF's low performance (0.40 on ORL, 0.38 on Complex) compared to other methods like PCA and LDA.

These references provide a theoretical and practical foundation for our project, validating the choice of linear algebra methods and offering insights into their strengths and weaknesses in face recognition tasks.

Conclusion

In this project, we investigated face recognition using traditional linear algebra techniques (PCA, LDA, PCA+LDA, NMF, SVD) and modern deep learning methods (CNN, ResNet50+kNN) across two datasets. The ORL Database results show linear algebra methods leading, with LDA at 1.0 (with Euclidean), PCA at 0.98, and PCA+LDA at 0.95, though ResNet+kNN matches PCA+LDA at 0.95, outperforming CNN (0.85) and NMF (0.40), benefiting from controlled conditions but at higher execution times for linear algebra methods (e.g., PCA: 40 seconds, LDA: 50 seconds). ResNet+kNN's 20-second execution time offers a good balance in controlled settings. On the Complex Dataset, deep learning methods excel (CNN: 0.67, ResNet+kNN: 0.65 vs. PCA+LDA: 0.52, PCA: 0.35), confirming their suitability for complex, real-world scenarios, with execution times of 10 seconds for CNN and 30 seconds for ResNet+kNN. Execution times on the Complex Dataset are higher for linear algebra methods (e.g., PCA: 130 seconds, PCA+LDA: 140 seconds), reflecting the dataset's size and variability. NMF and SVD offer the fastest execution (2–10 seconds) across both datasets, though NMF's low accuracy on the ORL Database limits its practicality. These findings highlight the practical value of linear algebra for high-accuracy face recognition in controlled settings (with LDA and PCA as top performers), while deep learning (particularly CNN and ResNet+kNN) remains preferable for complex scenarios, with ResNet+kNN offering a balanced option in controlled settings, and execution time trade-offs guiding method selection.

References

[1] Pentland, A., & Moghaddam, B. (2005). View-Based and Modular Eigenspaces for Face Recognition. *International Journal of Pattern Recognition and Artificial Intelligence*. Available at: `https://www.cs.cmu.edu/~changbo/publications/IJPRAI05.pdf`

[2] Wyant, T., & Lonce, W. (2003). Face Recognition with Non-negative Matrix Factorization. Available at: `https://lonce.org/Publications/publications/1838_1_FaceRecognitionWithNMF.pdf`